Path Planning Algorithms for an Autonomous Vehicle

MATLAB academic project | ELEN 331, Santa Clara University | Report and presentation: March 2024

Project status: Two-dimensional simulation experiments, with static-obstacle RRT, a moving-obstacle route-confidence scenario, and interactive A* maze search. The work is not a deployed autonomous-driving stack.

1) Overview and objective

The project investigates how a planner can find a route from a start location to a goal while responding to obstacle geometry and uncertainty about another object's motion. RRT explores free space by randomized tree growth. A* searches a discretized map using accumulated and estimated remaining costs. The moving-truck example connects candidate motion hypotheses to spatial constraints used by repeated planning calls.

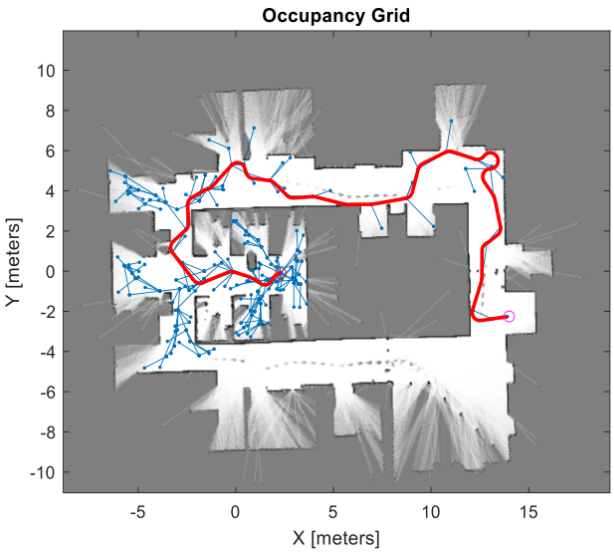


Figure 1. Occupancy-grid RRT example from report page 2. Blue branches and the red displayed route illustrate the separate occupancy-grid demonstration.

The author created/evaluated MATLAB scenarios, route-confidence logic, and visualizations. The A* file retains MathWorks demo attribution and is described as a reused/adapted implementation for maze experiments. The current résumé also discusses later heuristic/bidirectional RRT ideas for possible stereo-VSLAM integration; those extensions are not implemented by the two scripts in this ZIP.

2) Simulation environments and source files

Material	Content and role
RRT_Algorithm_Code1_Amarasinghe.m	Static circular-obstacle RRT, moving-truck setup, route-confidence update, and local RRTPredict function.
Astar_Algorithm_Code_Amarasinghe.m	Interactive 10 x 10 grid demo with target/obstacle/start selection, OPEN/CLOSED lists, and helper functions.
Final Project Amarasinge.pdf	13-page report with occupancy-grid example, circular-obstacle plots, truck-route sequence, epsilon examples, and A* maze figures.
Project Presentation Amarasinghe.pdf	Presentation dated March 20, 2024, with algorithm introductions and visual examples.

Static environment

The custom static example uses a normalized [0,1] x [0,1] workspace, start [0,0.8], and goal [0.95,0.1]. Four circular obstacles have centers/radii: [0.4,0.8]/0.2, [0.8,0.5]/0.1, [0.4,0.35]/0.15, and [0.1,0.2]/0.1. Obstacles are labelled House, Rock, Shop, and Tree for visualization.

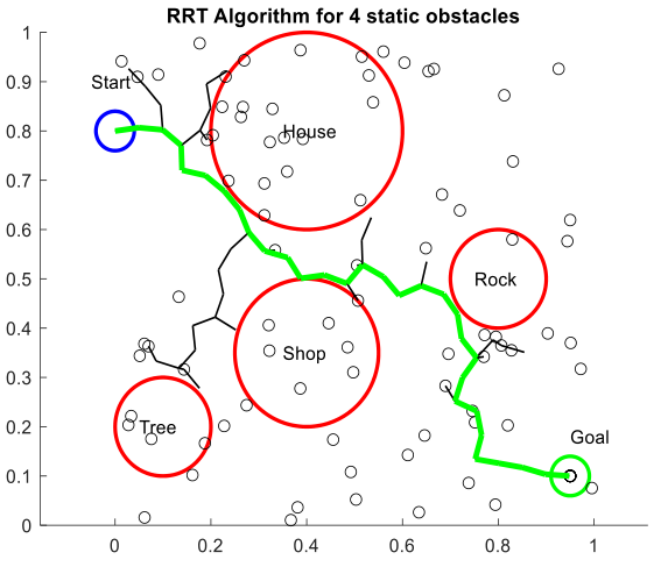


Figure 2. Static circular-obstacle scene and extracted green route. Source: report page 3.

These normalized scenario coordinates should not be confused with the meter-labelled occupancy grid in Figure 1. The report contains that additional illustration, while the supplied custom RRT script implements the circular-obstacle scenarios.

3) Static RRT implementation

Tree expansion

The tree begins with a start node and parent index zero. On each iteration, the script samples uniformly within the workspace with probability 0.9 and samples the goal otherwise. It computes Euclidean distances from every existing node to the sample and selects the nearest one. The new node is either the sample itself or a step of length epsilon toward it.

Parameter or operation	Submitted static example
Maximum iterations	1,000
Extension step epsilon	0.05
Goal-neighborhood radius	0.04
Random / goal sampling	90% / 10%
Nearest neighbor	Linear scan through the node list using Euclidean norm.
Obstacle rejection	Reject new endpoint inside any circular obstacle.
Route extraction	Parent tracing from the tree node closest to goal.

Parent links and visualization

Each accepted node stores its parent index. Black lines show accepted tree edges, while the extracted path is highlighted in green. This representation makes the exploration process inspectable without requiring a vehicle dynamics model. Random sampling can produce different routes between runs unless the random seed is fixed.

Collision and success interpretation

The script checks candidate points, not whole connecting segments, so a segment can cross an obstacle even when its endpoint is outside it. No vehicle footprint or turning constraint is included. The iteration loop can stop near the goal, but after exhausting the budget the code still traces the closest available node. A plotted route is therefore not by itself an explicit successful arrival.

A practical extension would add segment-level collision tests, obstacle inflation for robot dimensions, a validated final connection, and explicit success/failure status before returning a path. These are recommended improvements, not changes made to the supplied scripts.

4) Truck-route sequence: initial uncertainty

The dynamic scenario uses vehicle start [-0.15,0.5] and goal [1,0.5], three static circles, and a truck beginning at [0.25,0.5]. Candidate truck destinations are End1 [0.25,0.9], End2 [0.25,0.1], and End3 [0.7,0.2]. The vehicle start lies outside the [0,1] random-sampling square used by the planner.

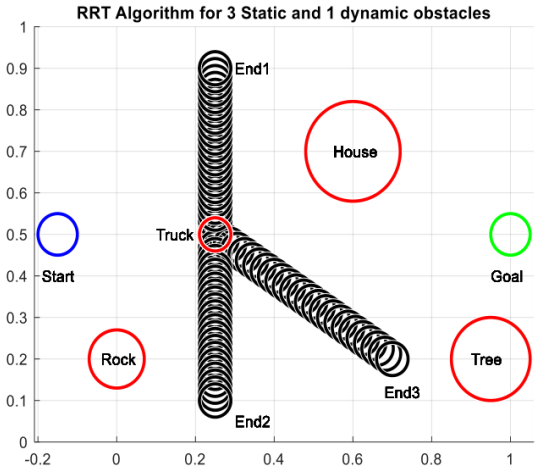


Figure 3. Initial scene with three possible truck-route corridors. Source: report page 4.

The code initializes trust to [1/3,1/3,1/3]. Black circles visualize possible future truck positions along all three routes. The selected route is chosen by randi([1,3]); a commented assignment allows a fixed choice. Twenty-five linearly interpolated positions define the observed truck sequence, with iteration starting at the second sample.

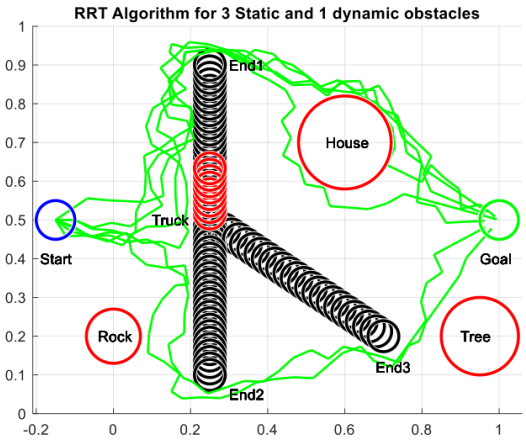


Figure 4. Early movement toward End1; red circles show observed truck positions and green lines show candidate vehicle paths. Source: upper plot, report page 5.

At each new truck position, the simulation recomputes beliefs and calls RRTPredict. The planner is restarted from the same carStart every time: these green overlays are separate candidate routes, not the path of an ego vehicle moving through time.

5) Truck-route sequence: crossing and final path

RRTPredict constructs 20 sample positions along each candidate truck route. When trust(i) exceeds threshold=0.28, it rejects candidate RRT endpoints within rtruck=0.04 of those positions. If a route's belief is at or below the threshold, that route does not contribute its exclusion corridor. Static circular obstacles are checked separately.

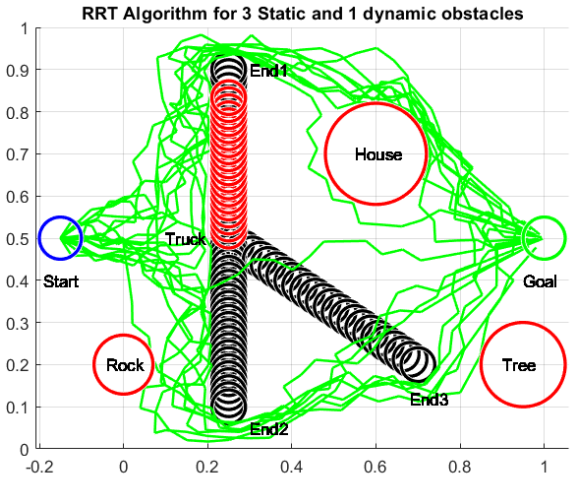


Figure 5. The report's next stage shows vehicle candidates crossing lower-belief corridors while avoiding the higher-belief truck route. Source: lower plot, report page 5.

This rule links uncertainty to usable space: the vehicle planner initially reserves several plausible corridors and can later relax alternatives. The threshold is a scenario tuning value, not an empirically calibrated collision probability. Exclusion applies to spatial route samples, not to a continuous time-dependent occupied region.

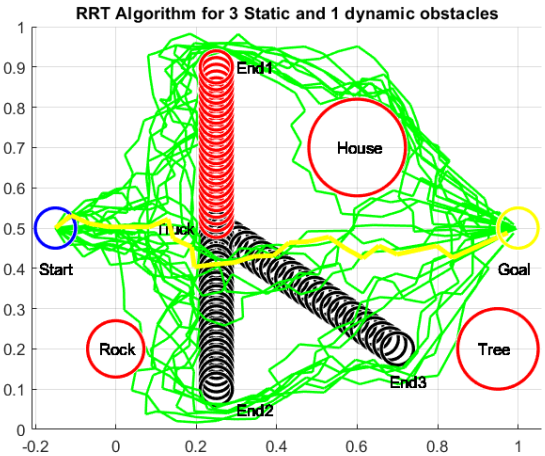


Figure 6. Final End1 example. A separate last RRTPredict call highlights a route in yellow; earlier candidate paths remain green. Source: upper plot, report page 6.

The final call uses the last confidence vector but generates a fresh random tree. The yellow goal circle is drawn separately, so its presence does not certify that every extracted route reaches it. The endpoint/edge-validation limitations of the static example also apply here.

6) Alternative routes and confidence calculation

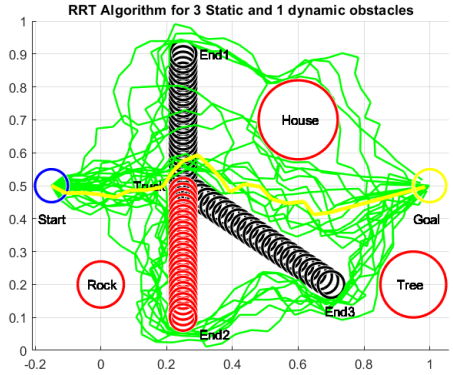


Figure 7. Truck selects the lower End2 route. Source: lower plot, report page 6.

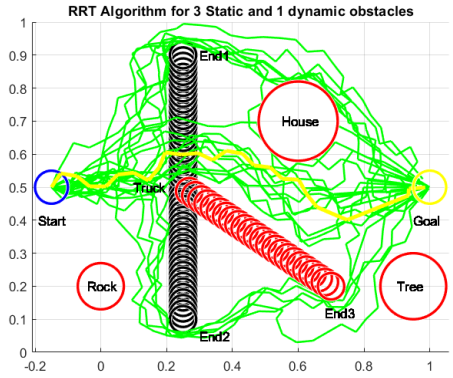


Figure 8. Truck selects the diagonal End3 route. Source: report page 7.

Implemented probability calculation

For candidate destination i, let d_i be the distance from the current truck position to that destination. The script sets R_i=-d_i and computes L_i=exp(0.75 R_i)/sum_j exp(0.75 R_j). It then applies trust_i=(L_i prior_i)/sum_j(L_j prior_j), with prior_i fixed at 1/3 each update.

Because the prior is uniform and freshly reset, the final trust values equal the normalized distance-based scores. This is Bayesian-style normalization of current geometric evidence, not recursive Bayesian filtering across the full observation history. The code does not feed the previous posterior back as the next prior; distance-ratio variables dis1/dis2/dis3 and prePosition are calculated/stored but do not determine the active confidence update.

The examples illustrate a planning policy under alternative truck destinations. They do not measure prediction accuracy, compare a learned intent model, or establish a calibrated probability of safe crossing.

7) Step-size sensitivity and RRT observations

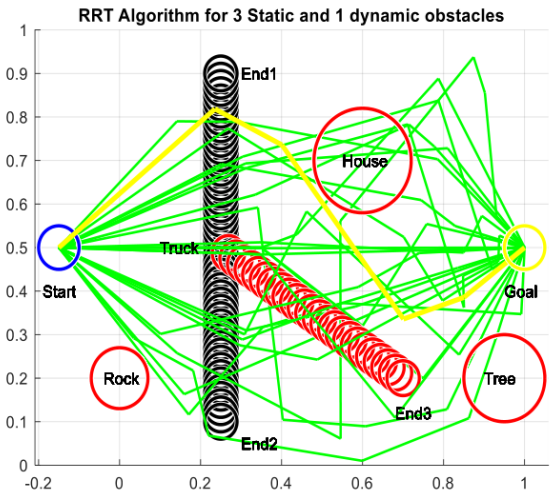


Figure 9. Reported large-epsilon illustration with coarse connections. Source: upper plot, report page 8.

A larger extension step allows long jumps across the workspace but makes endpoint-only collision checking especially weak. The report describes poor behavior with an overly large epsilon. Exact alternate values and repeated-trial statistics are not recorded, so the figure supports a qualitative parameter-sensitivity discussion.

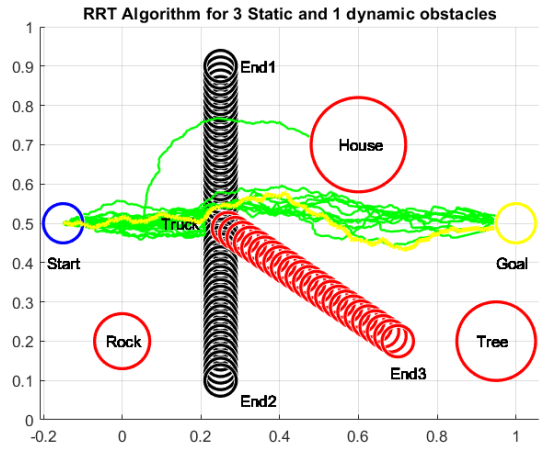


Figure 10. Reported small-epsilon example with limited progress. Source: lower plot, report page 8.

A small extension step can require many accepted samples to cover the same space. With a fixed iteration budget, exploration may remain local or fail to approach the goal. The report also notes narrow-gap difficulties and route-length/runtime variability in its tested implementation.

Those observations do not prove that all RRT implementations fail in narrow passages or that A* universally dominates RRT. Planner behavior depends on representation, sampling, validity checking, tuning, and the task. The uploaded code fixes epsilon=0.05; the alternate figures are retained as report examples rather than newly executed tests.

8) A* grid-search implementation

The interactive A* script defines MAX_X=MAX_Y=10 and represents cells by integer labels: obstacle=-1, target=0, robot=1, and free=2. Mouse input selects the target, obstacle cells, and initial vehicle cell. Eight-connected expansion examines horizontal, vertical, and diagonal neighbors within the array bounds.

Search element	Role
g(n)	Accumulated Euclidean movement cost to the candidate node.
h(n)	Euclidean distance estimate from candidate to target.
f(n)	g(n)+h(n), used for open-node selection.
OPEN	Candidate state, coordinates, parent, and cost values.
CLOSED	Obstacles and expanded/visited cells excluded from future neighbors.
Parent tracing	Reconstructs the selected route from target toward start.

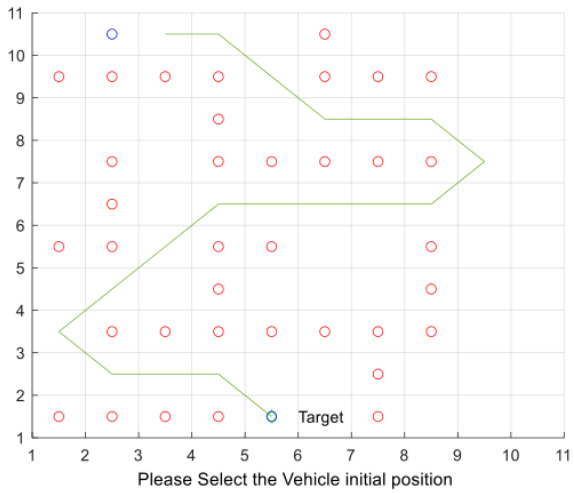


Figure 11. User-defined maze and displayed A* route. Source: report page 10.

The helper functions expand_array, insert_open, min_fn, node_index, and distance provide neighbor generation, list management, selection, and backtracking. The main loop updates candidate parents/costs and terminates at the target or when no OPEN node remains. A simple animation moves a plotted point along the returned sequence.

The file retains a MathWorks demo copyright header, including helper attribution. The author’s project contribution is described as using/adapting the implementation, constructing scenarios, observing outcomes, and documenting the comparison. The entire A* source is not presented as original algorithm authorship.

9) Maze evidence and evaluation boundaries

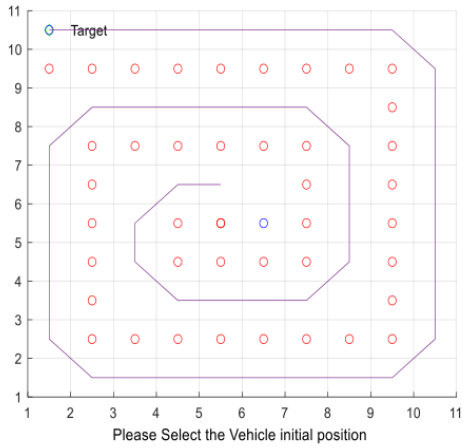


Figure 12. Spiral-like obstacle pattern. Source: report page 11.

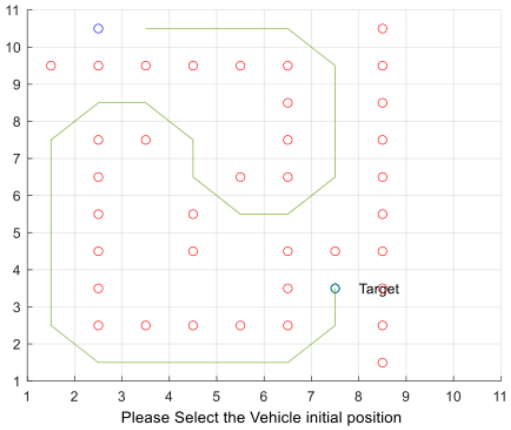


Figure 13. Curved obstacle layout and returned route. Source: report page 11.

The maze examples demonstrate route generation in several constructed cell layouts. The script's movement model allows diagonal steps but does not explicitly check whether diagonals cut across blocked adjacent cells. Vehicle shape, curvature constraints, dynamics, and continuous-space clearance are not included.

The report describes favorable A* behavior and shortest-route results in these experiments. No systematic path-length or timing benchmark is supplied. A shortest-path claim applies only under the selected graph, costs, heuristic, and valid-transition implementation; it is not a guarantee of the shortest physically drivable route.

The interactive input path also assumes valid clicks and indices. A reproducible evaluation package would replace manual-only setup with stored map cases and include blocked-goal, no-path, boundary, and diagonal-corner tests.

10) Conclusions, future work, and sources

Engineering interpretation

RRT demonstrates randomized exploration and the influence of step size and iteration budget. A* demonstrates explicit cost-driven search on a discrete map. The truck example shows how route confidence can change spatial constraints and trigger replanning. Together, these experiments connect algorithm structure, representation, uncertainty assumptions, and visual outcomes.

Recommended validation plan

Use fixed seeds and repeated RRT runs; require explicit goal reachability; validate every segment; inflate obstacles by vehicle footprint; prevent A* corner cutting; and store deterministic maps. Measure success rate, planning time, path length, number of samples/expanded nodes, and minimum clearance. Compare the planners on matched tasks rather than different representations alone.

For the moving-obstacle scenario, maintain time-indexed states, carry posterior beliefs forward if a recursive estimator is intended, justify observation likelihoods, and assess calibration on held-out trajectories. A closed-loop experiment would move the ego vehicle and verify collision clearance through time instead of repeatedly plotting routes from the same start.

Relation to later robotics work

The latest résumé mentions later exploration of heuristic costs and bidirectional RRT growth for potential ZED-based VSLAM integration. This description confines implemented details to the supplied MATLAB archive. Those later ideas remain separate unless supporting code and results are supplied.

Source register

Path Planning Algorithms for an Autonomous Vehicle.zip: RRT_Algorithm_Code1_Amarasinghe.m; Astar_Algorithm_Code_Amarasinghe.m; Final Project Amarasinge.pdf (13 pages); Project Presentation Amarasinghe.pdf. Truck-stage visuals follow report pages 4-8; maze figures follow pages 10-11. Résumé: Dak Amarasinghe Resume(20261030-094031).pdf. Existing-source attribution is retained from the A* file.

Preparation method: report/presentation, source-code, and figure inspection. No new MATLAB execution or benchmark was performed for this document.